

光储充微电网物联网平台

技术方案

dgiot_lite • DG-IoT 轻量联动 • 独立运行 • 源码交付

版本号: V1.0

文档状态: 初稿

日期: 2026 年 6 月

迪格 (杭州) 物联科技有限公司

目录

1	项目概述	2
1.1	项目背景	2
1.2	建设目标	2
1.3	项目范围与分工	2
2	技术选型	3
2.1	选型策略	3
2.2	技术栈	3
3	总体架构设计	3
3.1	数据流向	4
4	数据采集协议方案	4
4.1	Modbus RTU/TCP	4
4.2	IEC 60870-5-104	4
4.3	OPC UA	4
5	应用功能设计	5
5.1	设备与点位管理	5
5.2	2D 组态监控	5
5.3	碳排优化细则	5
5.4	关键 KPI	6
5.5	告警管理	6
5.6	数据推送	6
6	数据存储方案	6
6.1	TDengine 时序存储	6
6.2	PostgreSQL 关系存储	6
6.3	降级模式	7
7	部署方案	7
7.1	服务器配置 (≤500 设备)	7
7.2	Docker 一键部署	7
7.3	裸机部署	7
7.4	数据库说明	7

8	实施计划	8
8.1	交付物	8
9	运维服务方案	8
10	投资概算（脱敏版）	9
10.1	费用构成	9
10.2	DG-IoT 联动（可选）	9
10.3	付款方式建议	10

1 项目概述

1.1 项目背景

客户需要一套可源码交付的物联网平台，用于光储充微电网场站的数据采集与监控。客户拿到源码后自行掌握二次开发、部署和运维。

我方自有物联网平台与本项目独立——本项目交付的是基于 Python 全开源技术栈的轻量级物联网平台，专为满足客户源码交付需求而构建。

1.2 建设目标

1. 四协议统一接入：Modbus RTU、Modbus TCP、IEC 104、OPC UA 实际跑通
2. 时序 + 关系双存储：TDengine（时序）+ PostgreSQL（关系），开源免费
3. 2D 组态监控：HTML5 Canvas 实现，浏览器直接访问，无需插件
4. 数据推送：MQTT / HTTP 实时推送至第三方应用
5. 源码交付：完整 Python 源码，客户可自行编译部署和二次开发

1.3 项目范围与分工

我方负责物联网平台层——平台部署、协议适配、组态配置、数据推送，确保系统实际跑通。

工作内容	我方（平台层）	客户（应用层）
数据采集引擎	4 协议采集，实际跑通	—
2D 组态	组态页面 + 数据绑定	业务画面定制
数据库	TDengine+PG 部署 + 表结构	—
数据推送	MQTT/HTTP 通道开发	第三方对接
碳排优化	数据采集 + 核算模型	策略制定
源码处置	—	客户自行处置

2 技术选型

2.1 选型策略

本项目定位为轻量学习版，预算约 5 万元，技术选型原则：

- 全开源：所有组件开源免费，无 License 成本
- Python 优先：生态丰富、开发高效、客户易于接手
- 异步架构：asyncio + FastAPI，支撑多设备并发采集
- 降级模式：无 PostgreSQL/TDengine 时自动降级为 SQLite，单机即跑

2.2 技术栈

层次	选型	说明
核心语言	Python 3.10+	异步非阻塞，生态丰富
Web 框架	FastAPI + uvicorn	自动 API 文档，高性能异步
Modbus	pymodbus 3.6+	RTU + TCP 双模式
IEC 104	自研轻量客户端	ASDU 解析，总召 + 变化传输
OPC UA	opcua-asyncio	订阅 + 轮询双模式
时序存储	TDengine 3.x	列式存储，10-20:1 压缩
关系存储	PostgreSQL 15+	SQLAlchemy async ORM
消息推送	paho-mqtt + httpx	MQTT+HTTP 双通道
前端	HTML5 Canvas + Vanilla JS	零依赖 2D 组态
部署	Docker Compose	一键部署全部组件

3 总体架构设计

系统采用四层架构：

1. 设备接入层：协议适配器（Modbus RTU/TCP、IEC 104、OPC UA），每个设备一个适配器实例，独立异步采集循环
2. 核心服务层：采集调度引擎（CollectorEngine）+ 告警引擎（AlarmEngine）+ 推送引擎（PushEngine），通过回调链串联

3. 数据存储层：TDengine 存遥测时序数据，PostgreSQL 存设备档案/配置/告警，降级模式自动切 SQLite
4. Web 服务层：FastAPI REST API + WebSocket 实时推送 + 2D 组态静态页面

3.1 数据流向

采集流程：调度器定时触发 → 协议适配器读取硬件 → 解析为 PointValue → 并行写入 TDengine → 转发告警引擎和推送引擎 → WebSocket 广播至前端

4 数据采集协议方案

4.1 Modbus RTU/TCP

基于 pymodbus 异步客户端实现。RTU 模式通过 RS-485 串口轮询，TCP 模式通过以太网并发连接。支持功能码 01-04、06、16，自动解析 int16/uint16/int32/float32/float64 数据类型。每个设备独立采集周期（1-60 秒可配），采集失败自动重连。

4.2 IEC 60870-5-104

轻量自研 IEC 104 客户端（主站模式），实现：

- APCI 帧解析（I 帧发送序列号管理、U 帧 STARTDT/STOPDT）
- ASDU 类型支持：M_ME_NA_1(9)、M_ME_NB_1(11)、M_ME_NC_1(13)、M_SP_NA_1(1)、M_DP_NA_1(3)
- 总召命令（C_IC_NA_1）+ 时钟同步（C_CS_NA_1）
- TCP/2404 连接，自动重连

4.3 OPC UA

基于 opcua-asyncio，支持：

- Browse 地址空间自动发现变量节点
- 订阅（Subscription）模式：数据变化主动推送
- 轮询（Read）模式：周期读取
- 安全策略 None / Basic256Sha256 + 用户名密码认证

5 应用功能设计

5.1 设备与点位管理

- 设备档案：设备名称、类型（光伏/储能/充电桩/电表/传感器）、协议、通讯参数
- 点位配置：协议地址、数据类型、采集周期、缩放系数、告警阈值
- 批量导入：支持 Excel/CSV 点位模板批量导入

5.2 2D 组态监控

基于 HTML5 Canvas 的 2D 组态画面：

- 画面：场站总览、光伏监控、储能监控、充电桩监控、碳排仪表盘、告警总览（6 画面）
- 图元：光伏阵列、逆变器、PCS、变压器、充电桩、电网接口等电力设备图标
- 实时交互：点击设备弹出数据详情面板，告警变色闪烁，WebSocket 实时刷新

5.3 碳排优化细则

碳排放源识别：光储充场站无直接排放源，间接排放来自电网购电（范围二）。碳减排来自光伏替代电网电量和充电桩替代燃油。

数据采集要求：从逆变器采集光伏发电量（15 分钟）、PCS 采集充放电量（15 分钟）、关口表采集电网购售电量（15 分钟）、充电桩采集服务电量（累计）、辐照仪采集辐照度（1 分钟）。

核算标准：采用 GB/T 32150 方法学，电网排放因子引用生态环境部最新发布值。
净碳排放 = 电网购电量 × 排放因子 - 光伏发电量 × 排放因子 - 充电量 × 燃油替代因子

报告周期：实时仪表盘（1 分钟刷新）+ 日报 + 月报 + 年报

5.4 关键 KPI

类别	指标	目标值	周期
采集质量	数据完整率	$\geq 99\%$	日/月
采集质量	设备在线率	$\geq 99\%$	日/月
采集质量	平均采集延迟	≤ 3 秒	实时
平台性能	数据推送延迟	≤ 1 秒	实时
平台性能	API 响应时间	$\leq 500\text{ms}$	实时
碳排指标	光伏碳减排量	按装机核算	月/年
碳排指标	场站净碳排放	≤ 0 (碳中和)	月/年
碳排指标	可再生能源占比	$\geq 80\%$	月

5.5 告警管理

阈值告警（高限/上上限/低限/下下限）、变化率告警、设备离线告警。三级分类 P0/P1/P2。WebSocket 实时推送至前端 + MQTT 推送至第三方。

5.6 数据推送

MQTT（默认）+ HTTP POST 双通道。消息格式 JSON，规则引擎按设备类型/点位过滤。断网本地缓存、恢复补推。

6 数据存储方案

6.1 TDengine 时序存储

超级表 device_telemetry (ts + value + quality)，TAGs (device_id, point_id, point_name, unit, device_type)。每个设备点位独立子表。列式存储 + 差值编码，压缩比 10-20:1。1 分钟级数据保留 12 个月。

6.2 PostgreSQL 关系存储

存储设备档案 (devices)、点位配置 (data_points)、告警记录 (alarm_records)、推送目标 (push_targets)。SQLAlchemy async ORM + Alembic 迁移。

6.3 降级模式

无 TDengine/PostgreSQL 环境时，自动降级为 SQLite 存储全部数据，单机即可完整运行——适合开发测试和轻量部署场景。

7 部署方案

7.1 服务器配置（≤500 设备）

资源	最低	推荐
CPU	4 核	8 核
内存	8 GB	16 GB
磁盘	200 GB SSD	500 GB SSD
系统	Ubuntu 20.04+	Ubuntu 22.04 LTS

7.2 Docker 一键部署

```
cd scripts && docker-compose up -d
```

一键启动 IoT 平台 + PostgreSQL + TDengine + EMQ X 四个容器。

7.3 裸机部署

```
pip install -r requirements.txt
python scripts/init_db.py
python run.py
# 访问 http://localhost:8000/docs
```

7.4 数据库说明

数据库采用开源免费版本：TDengine Community（AGPL v3）+ PostgreSQL（PostgreSQL License）。客户按《部署手册》自行安装，不产生 License 费用。

8 实施计划

阶段	工作内容	工期	验收标准
P1	需求确认 + 环境准备	1 周	设备清单、通讯参数、服务器就绪
P2	平台部署 + 数据库初始化	1 周	平台可登录，数据库连接正常
P3	四协议适配 + 联调	3 周	各协议至少接入 1 台真实设备跑通
P4	2D 组态 + 数据推送	2 周	组态画面实时数据展示，推送链路正常
P5	联调测试 + 文档	2 周	端到端测试通过，数据准确率达标
P6	验收 + 培训	1 周	交付物齐全，客户签字验收
合计		10 周	

8.1 交付物

- IoT 平台完整 Python 源码（含 Git 仓库）
- 技术方案（本文档）+ 部署手册 + 协议接入指南 + API 文档
- 数据库初始化脚本 + Docker Compose 部署文件
- 验收测试报告

9 运维服务方案

服务范围:保障交付的 IoT 平台长期稳定运行——四协议采集链路、TDengine/PostgreSQL 数据库运行、数据推送链路、2D 组态可用性。

级别	响应	场景	方式
P0 紧急	≤30min	平台宕机、全站数据中断	远程 + 电话
P1 重要	≤2h	单协议中断、推送断开	远程
P2 一般	≤8h	个别设备离线	工单

不含：平台版本升级（交付版本长期可用，升级迭代不纳入本项目）；客户应用层运维；服务器硬件。

10 投资概算（脱敏版）

10.1 费用构成

序号	费用项目	计价	说明
1	四协议适配开发	按工作量	Modbus RTU/TCP + IEC104 + OPCUA 采集引擎
2	2D 组态开发	6 个画面	总览/光伏/储能/充电桩/碳排/告警
3	数据推送开发	3 通道	MQTT + HTTP + Web-Socket
4	数据库部署	2 个	TDengine + PostgreSQL 建库建表
5	平台部署 + 联调	按人天	环境部署 + 协议调试 + 性能调优
6	文档 + 培训	按项	部署手册 + API 文档 + 培训
合计			详见商务报价函

说明：

- 全开源组件，无第三方 License 费用
- 数据库（TDengine Community + PostgreSQL）免费，客户自装
- 不含服务器硬件，客户自行采购
- 年度维保服务按年另签

10.2 DG-IoT 联动（可选）

dgiot_lite 为 DG-IoT 开源物联网平台（Erlang/OTP）的 Python 轻量联动版本。独立运行完整闭环的同时，可选推送数据至 DG-IoT 主平台：

- 联动方式：MQTT 物模型格式推送 → DG-IoT 规则引擎自动接收

- 配置方式：管理后台添加推送目标，类型选择 dgiot
- 零侵入：不配置则完全不依赖 DG-IoT，独立运行
- 数据格式：自动转换为 DG-IoT 物模型标准格式（properties + timestamp）

10.3 付款方式建议

合同签订后 → 平台部署完成 → 联调测试通过 → 终验合格，分四期支付。